

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»
(СПбГУТ)

АРХАНГЕЛЬСКИЙ КОЛЛЕДЖ ТЕЛЕКОММУНИКАЦИЙ
ИМ. Б.Л. РОЗИНГА (ФИЛИАЛ) СПбГУТ
(АКТ (Ф) СПбГУТ)

КУРСОВОЙ ПРОЕКТ

НА ТЕМУ

РАЗРАБОТКА ПОДСИСТЕМЫ

«ИНТИ. УЧЁТ ПРОЕКТОВ»

Л109. 25КП01. 011 ПЗ

(Обозначение документа)

МДК.02.01 Технология разработки

программного обеспечения

Студент	ИСПП-21	08.12.2025	К.С. Ключанов
	(Группа)	(Подпись)	(И.О. Фамилия)
Преподаватель		08.12.2025	Ю.С. Маломан
	(Подпись)	(Дата)	(И.О. Фамилия)

Архангельск 2025

СОДЕРЖАНИЕ

Перечень сокращений и обозначений	3
Введение.....	4
1 Сбор и анализ требований.....	6
1.1 Назначение и область применения.....	6
1.2 Постановка задачи	6
1.3 Выбор состава программных средств.....	8
2 Проектирование программного обеспечения.....	10
2.1 Проектирование интерфейса пользователя	10
2.2 Разработка архитектуры программного обеспечения	11
2.3 Проектирование базы данных	12
3 Разработка и интеграция модулей программного обеспечения	13
3.1 Разработка программных модулей	13
3.2 Реализация интерфейса пользователя.....	16
3.3 Разграничение прав доступа пользователей.....	188
3.4 Экспорт и импорт данных	20
4 Тестирование и отладка программного обеспечения	233
4.1 Структурное тестирование.....	233
4.2 Функциональное тестирование	23
5 Инструкция по эксплуатации программного обеспечения.....	255
5.1 Установка программного обеспечения	255
5.2 Инструкция по работе.....	266
Заключение	29
Список использованных источников	29

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящем курсовом проекте применяются следующие сокращения и обозначения:

БД – база данных

ОС – операционная система

ПК – персональный компьютер

ПО – программное обеспечение

СУБД – система управления базами данных

API – интерфейс прикладного программирования

ERD – диаграмма взаимосвязей сущностей

HTTP – протокол передачи гипертекста

ID – идентификатор

IDE – интегрированная среда разработки

WPF – фреймворк для создания графических пользовательских интерфейсов

ВВЕДЕНИЕ

В современном мире цифровые технологии играют ключевую роль в оптимизации и автоматизации процессов управления проектами в различных отраслях. Информационные системы и веб-приложения позволяют эффективно собирать, хранить, фильтровать и отображать данные, обеспечивая высокую прозрачность и контроль над деятельностью. Это особенно актуально для организаций, занимающихся проектированием, строительством и другими видами деятельности, где управление множеством проектов, их участниками и статусами требует высокой точности и оперативности. Ручной ввод данных, ведение учёта в электронных таблицах или бумажных журналах приводит к снижению производительности, увеличению риска ошибок и затрудняет аналитику.

Для компаний, осуществляющих проектную деятельность, эффективное управление проектами является критически важным элементом успеха. Однако во многих организациях процессы учёта и мониторинга проектов до сих пор частично или полностью зависят от ручных операций, что замедляет принятие решений и снижает общую прозрачность проектного портфеля. Цифровая трансформация управления проектами позволяет не только автоматизировать рутинные задачи, но и предоставить менеджерам и администраторам удобный инструмент для анализа текущей ситуации с помощью фильтрации и поиска, планирования и контроля исполнения.

Таким образом, разработка клиент-серверной подсистемы управления проектами позволит автоматизировать процессы учёта, планирования и отслеживания статусов проектов, минимизировать человеческий фактор, обеспечить мгновенный доступ к актуальной информации и повысить общий уровень управляемости и эффективности.

Целью курсового проектирования является разработка программного решения для сотрудников компании, предназначенного для автоматизации управления проектами, включая учёт, фильтрацию, поиск и отображение

информации о проектах, их руководителях и статусах, с учётом ролевой модели пользователя.

Для достижения поставленной цели требуется решить следующие задачи:

- проанализировать предметную область;
- определить требования к разрабатываемому программному средству;
- спроектировать архитектуру системы, включающую серверную часть, базу данных и клиентское оконное приложение;
- реализовать серверную часть, обеспечивающую выполнение бизнес-логики приложения и разграничение прав доступа;
- разработать оконное приложение для менеджеров компании;
- обеспечить удобство взаимодействия пользователя с приложением за счёт оптимизации навигации и логики интерфейса;
- провести тестирование функциональности программного продукта.

Реализация указанных задач позволит создать удобный инструмент, который поможет организовать наглядный просмотр данных проектов и организует заполнение данных в структурированной форме, обеспечивая целостность и единообразие информации в централизованной базе данных.

1 Сбор и анализ требований

1.1 Назначение и область применения

Основным назначением подсистемы является оптимизация работы с данными о проектах организации и сохранением информации о новых проектах, что позволит снизить временные затраты на создание, фильтрацию и редактирование данных о проектах, а также позволит минимизировать человеческие ошибки.

Подсистема предназначена для использования в организациях, занимающимся учётом данных, например, в организации ООО «ИНТИ». Основными пользователями являются менеджеры по учёту проектов.

1.2 Постановка задачи

Требуется разработать оконное приложение, которое предоставит доступ к следующей функциональности:

- авторизация пользователей;
- просмотр, создание, редактирование и удаление данных о проектах компании;
- просмотр подробных данных отдельного проекта и его руководителя;
- добавление заметок о студентах;
- фильтрация и сортировка проектов;
- поиск проектов по их названию;
- импорт данных о проектах из csv-файла;
- экспорт данных о проектах в формате .pdf.

При запуске оконного приложения отображается страница авторизации. Пользователь может авторизоваться в системе, если у него имеются данные учётной записи, которые предоставляет организация. При успешной авторизации пользователь попадает на главную страницу приложения с

доступной функциональностью в зависимости от роли пользователя в подсистеме.

Гость имеет доступ к просмотру информации о проектах компании, поиску проекта по его названию, фильтрации и сортировки проектов. Также гость имеет доступ к прохождению авторизации.

Менеджер имеет доступ к функциональности гостя, а также имеет права на создание, редактирование, удаление данных проектов и просмотр подробностей о проекте и его руководителе.

Администратор имеет доступ к функциональности менеджера, а также имеет права на импорт и экспорт данных проектов.

На рисунке 1 изображена диаграмма вариантов использования приложения.

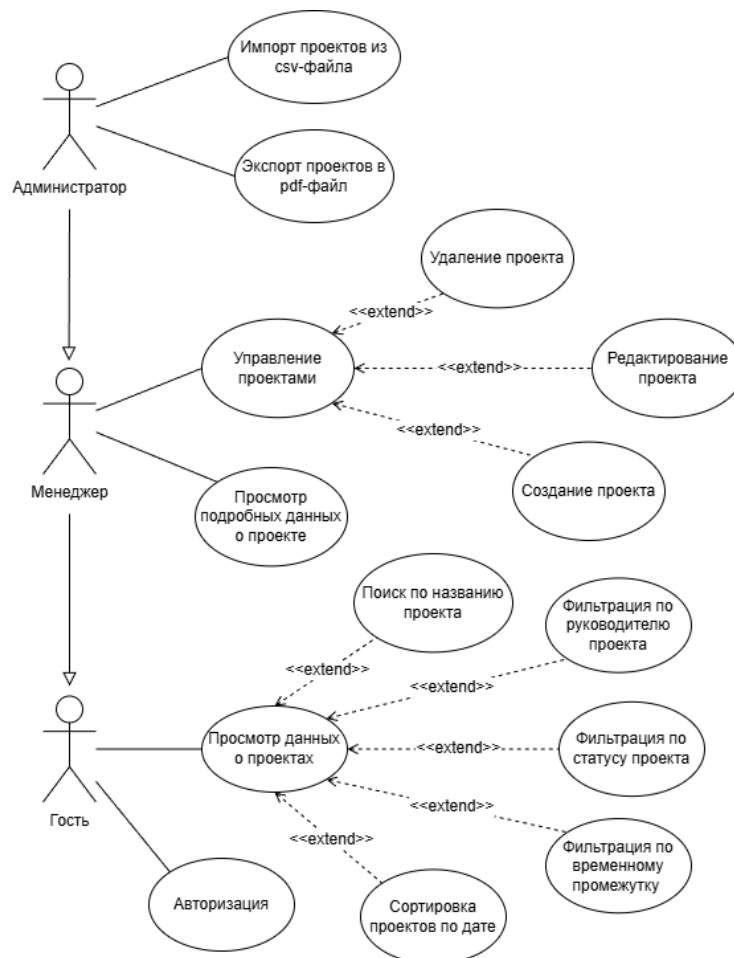


Рисунок 1 – Диаграмма вариантов использования приложения

1.3 Выбор состава программных средств

Согласно поставленной цели проекта требуется разработать оконное приложение для учёта проектов организации.

Работа с оконным приложением будет осуществляться на устройствах с подключением по локальной сети. Для работы требуется ПК с ОС Windows.

В качестве СУБД выбрана PostgreSQL 16, так как это надёжная, мощная и полнофункциональная система управления реляционными базами данных с поддержкой сложных запросов, транзакций и расширяемости. Выбрана как стабильное и масштабируемое решение для хранения структурированных данных проектов.

Клиентское приложение будет написано на языке программирования C#, так как C# – это объектно-ориентированный, простой и в то же время мощный язык программирования, который позволяет разработчикам создавать многофункциональные приложения и из-за большого разнообразия синтаксических конструкций.

Серверная логика реализуется на языке программирования Python 3.12 с использованием фреймворка FastAPI, который выбран за его высокую производительность, встроенную поддержку асинхронности, автоматическую генерацию документации API (Swagger UI, ReDoc) и удобную интеграцию с Pydantic для валидации данных.

Для разработки клиентской части подсистемы будет использоваться IDE Microsoft Visual Studio Community 2022, так как это быстрая, производительная и простая в освоении IDE. Она позволяет работать с различными языками программирования, имеет продвинутую систему IntelliCode, позволяющую автоматически завершать целые строки кода, определяющую многие типичные задачи и рекомендуемую правильное быстрое действие.

Для разработки серверной части подсистемы будет использоваться IDE Visual Studio Code, так как её кроссплатформенность, гибкость за счёт мощной системы расширений, лёгкость и скорость работы, поддержка множества

языков (в том числе Python), встроенные инструменты для отладки, интеграции с Git и интуитивно понятный интерфейс делают её мощным и настраиваемым редактором кода, который легко адаптируется под любые задачи разработки.

Для функционирования системы на стороне сервера достаточны следующие программные и технические средства:

- ОС Ubuntu Server версии 22.04 или аналоги;
- сервер БД: PostgreSQL версии не ниже 16;
- процессор с частотой 2 ГГц;
- свободная оперативная память объемом 4 ГБ;
- ПО для работы API: Python 3.12, FastAPI.

Для функционирования системы на стороне клиента достаточны следующие программные и технические средства:

- ОС Windows 10 и выше.
- процессор с частотой 2 ГГц;
- оперативная память: минимум 2 ГБ;
- постоянное интернет-подключение.

Приведенный набор программных средств обеспечивает эффективное функционирование оконного приложения для автоматизированного учёта проектов организации.

2 Проектирование программного обеспечения

2.1 Проектирование интерфейса пользователя

В рамках разработки оконного приложения при помощи инструмента draw.io спроектирован интерфейс пользователя в виде набора wireframe, который позволяет представить его основные элементы и функциональность. Wireframe страниц авторизации, списка проектов, просмотр подробностей о проекте и создание проекта представлен на рисунке 2.

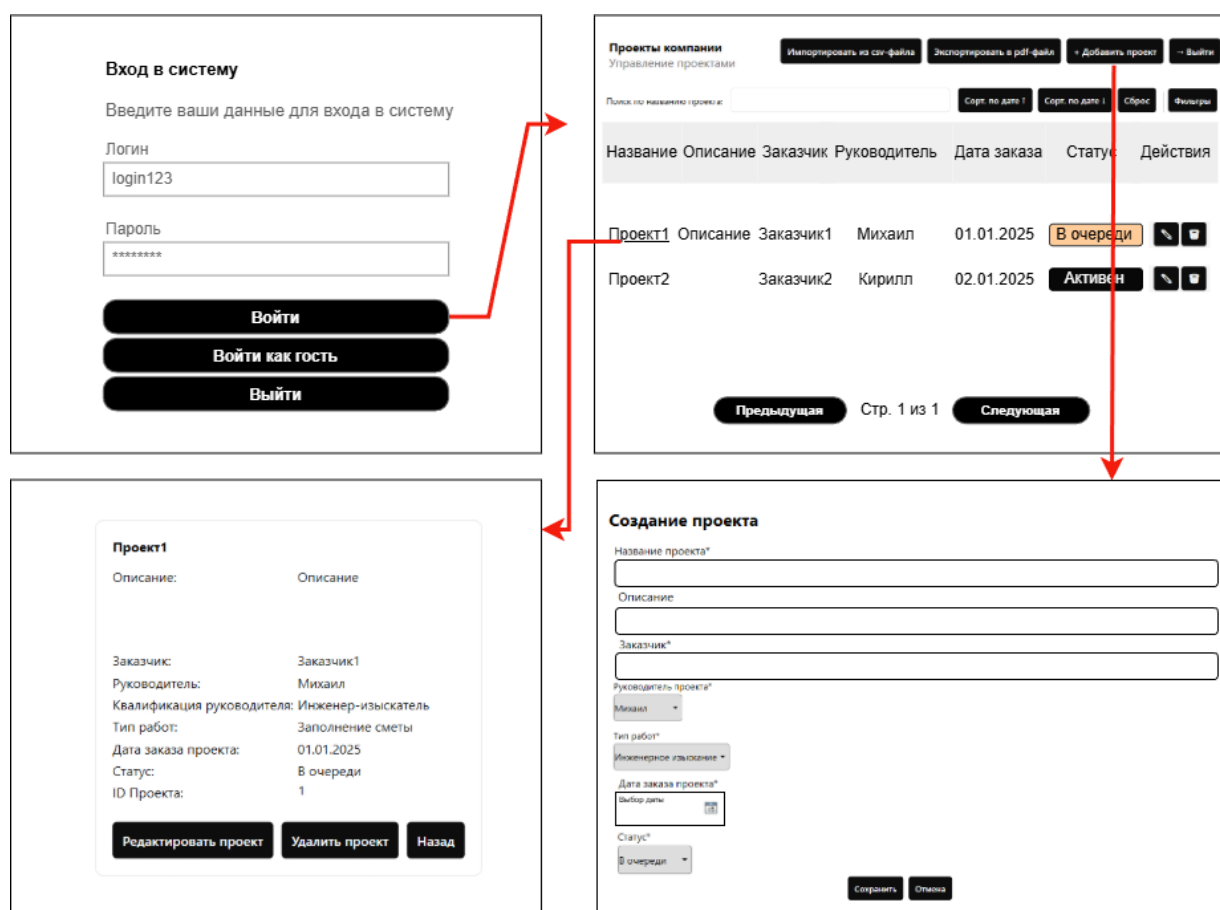


Рисунок 2 – draw.io. Wireframe основных страниц

Для оконного приложения выбраны следующие цвета:

- основной цвет: #0D0D0D;
- цвет фона: #FFFFFF;

- цвет фона столбцов таблицы: #F8F9FA;
- основной цвет текста: #0D0D0D;
- вторичный цвет текста: #757575;
- цвет текста ошибок: #F44336;

2.2 Разработка архитектуры программного обеспечения

Архитектура подсистемы построена на основе клиент-серверной модели и включает в себя следующие компоненты: оконное приложение, БД и API для взаимодействия с БД [5].

Диаграмма разворачивания компонентов изображена на рисунке 4.

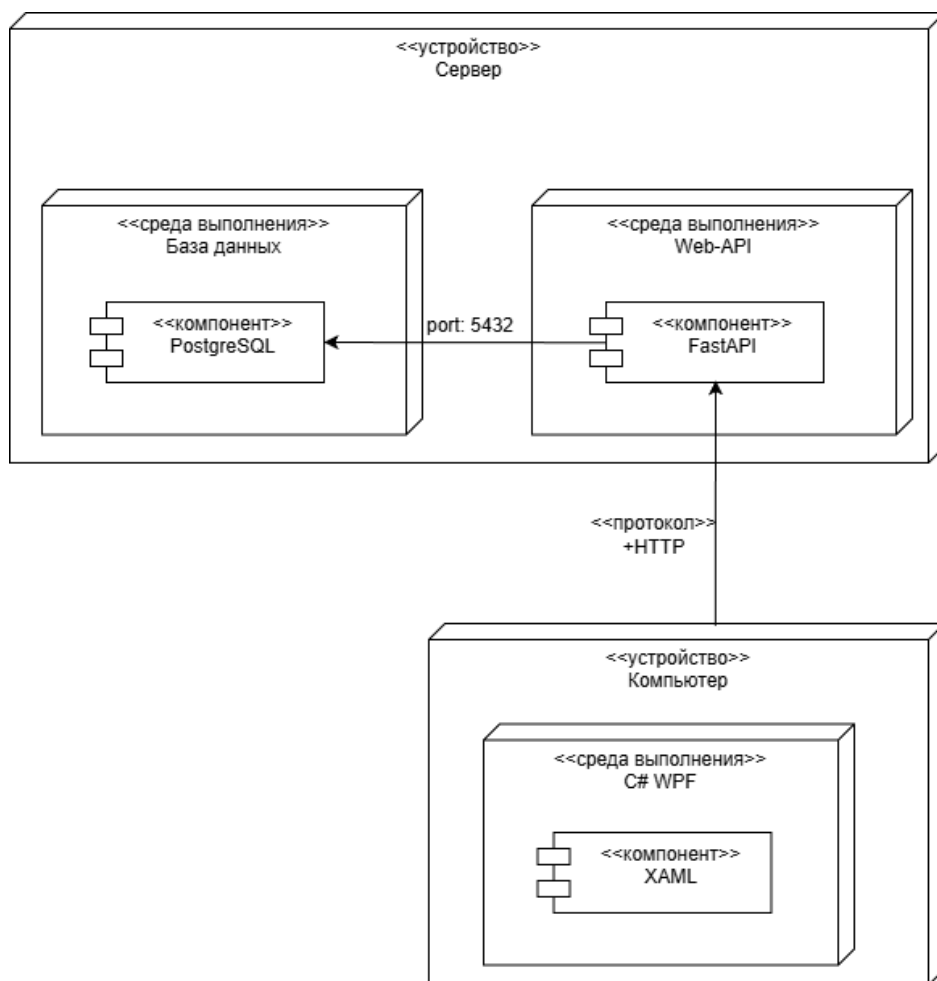


Рисунок 4 – Диаграмма разворачивания компонентов

2.3 Проектирование базы данных

В рамках разработки оконного приложения при помощи pgAdmin 4 спроектирована физическая модель БД для подсистемы учёта проектов организации ООО «ИНТИ», представленная на рисунке 3 в виде ERD [3].

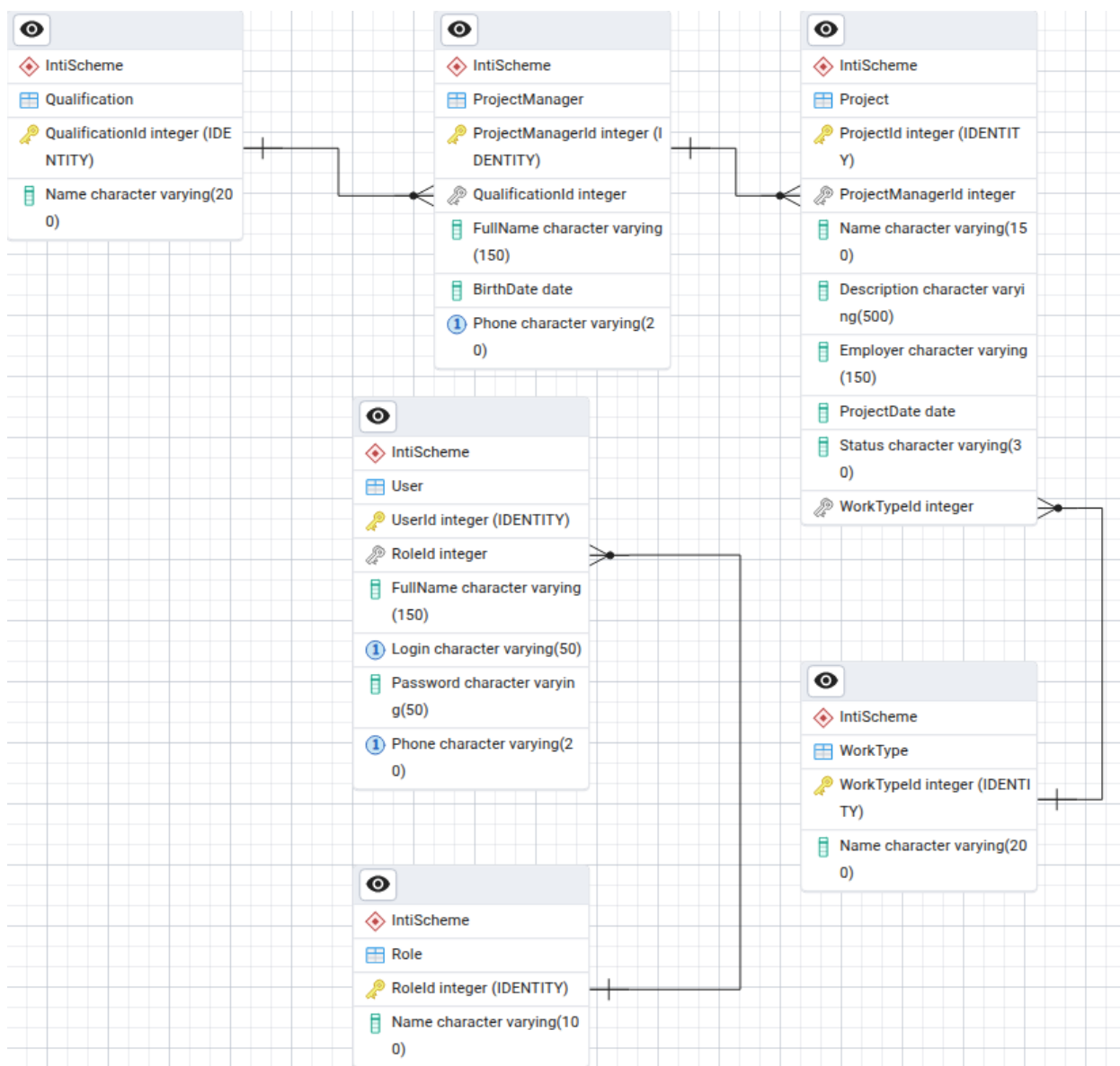


Рисунок 3 – pgAdmin 4. Физическая модель БД

3 Разработка и интеграция модулей программного обеспечения

3.1 Разработка программных модулей

В ходе курсового проектирования разработаны следующие приложения: клиентское приложение на C# в Visual Studio 2022 и Web-API приложение на Python в Visual Studio code [2].

Обращение к информации, хранящейся в БД, осуществляется посредством Web-API приложения, с использованием фреймворка FastAPI, созданными моделями таблиц БД. Код метода для получения списка проектов с фильтрацией и пагинацией представлен листингом 1.

Листинг 1 – Код метода для отправки GET-запроса на сервер

```
@app.post("/projects/details", response_model=PaginatedResponse)
def get_projects_details(
    filters: ProjectFilter,
    db: Session = Depends(get_db),
    page: int = Query(1, ge=1, description="Номер страницы")
):

    try:
        # Запрос для получения данных
        data_query = """SELECT ... FROM ..."""

        # Выполнение запроса на получение данных
        result = db.execute(text(data_query), params)
        projects_data = result.fetchall()

        # преобразование данных в модель
        projects = [
            ProjectDetails(
                ProjectNumber=project.project_number,
                ProjectName=project.project_name,
                Description=project.description,
                Employer=project.employer,
                ProjectManagerFullName=
                project.project_manager_full_name,
                ProjectManagerQualification=
                project.project_manager_qualification,
```

```

        WorkTypeName=project.work_type_name,
        ProjectDate=project.project_date,
        Status=project.status
    )
    for project in projects_data
]

return PaginatedResponse(
    projects=projects,
    total_count=total_count,
    page=page,
    total_pages=total_pages,
    has_next=page < total_pages,
    has_prev=page > 1
)

except Exception as e:
    logger.error(f"Ошибка при получении проектов: {str(e)}")
    raise HTTPException(status_code=500, detail=f"Ошибка при
получении проектов: {str(e)}")

```

Взаимодействие оконного приложения с сервером происходит при помощи HTTP-запросов к API, ответы возвращаются в формате JSON. Для получения списка проектов с фильтрацией и пагинацией был реализован метод, который отправляет POST-запрос к эндпоинту `/projects/details` и обрабатывает полученные данные, представленная листингом 2.

Листинг 2 – Код метода для получения списка проектов с фильтрацией и пагинацией

```

public async Task<PaginatedResponse?> GetProjectsAsync(
    int page = 1,
    string? projectStatus = null,
    string? projectManager = null,
    DateOnly? projectDateFrom = null,
    DateOnly? projectDateTo = null,
    string? projectSearch_text = null)
{
    // Формирование тела запроса с фильтрами
    var requestData = new
    {
        status = projectStatus,
        project_manager = projectManager,
        project_date_from = projectDateFrom?.ToString("yyyy-MM-
dd"),
        project_date_to = projectDateTo?.ToString("yyyy-MM-dd"),

```

```

        search_text = projectSearch_text
    };

    // Сериализация данных в JSON
    var json = JsonSerializer.Serialize(requestData);
    var content = new StringContent(json, Encoding.UTF8,
"application/json");

    // Отправка POST-запроса к API с указанием номера страницы
    var response = await
_httpClient.PostAsync($"projects/details?page={page}", content);
    if (response.IsSuccessStatusCode)
    {
        var responseContent = await
response.Content.ReadAsStringAsync();

        // Десериализация ответа в модель PaginatedResponse
        var paginatedResponse =
JsonSerializer.Deserialize<PaginatedResponse>(responseContent,
new JsonSerializerOptions { PropertyNameCaseInsensitive = true
});
        return paginatedResponse;
    }
    else
    {
        // В случае ошибки логируем содержимое ответа
        var errorContent = await
response.Content.ReadAsStringAsync();
        Console.WriteLine($"Error content: {errorContent}");
        return null;
    }
}

```

Для получения подробностей о проекте по его ID реализован метод Web-API, который представлен листингом 3.

Листинг 3 – Код получения подробностей о проекте по его ID

```

@app.get("/projects/{project_id}",
response_model=ProjectDetails)
def get_project_by_id(project_id: int, db: Session =
Depends(get_db)):
    """
    Получить детали проекта по его ID.
    """

    query_sql = """
    SELECT

```

```

        p."ProjectId" as project_number,
        p."Name" as project_name,
        p."Description" as description,
        p."Employer" as employer,
        pm."FullName" as project_manager_full_name,
        q."Name" as project_manager_qualification, --
Включаем квалификацию
        wt."Name" as work_type_name, --
Включаем тип работ
        p."ProjectDate" as project_date,
        p."Status" as status
    FROM "Project" p
    JOIN "ProjectManager" pm ON p."ProjectManagerId" =
pm."ProjectManagerId"
    JOIN "Qualification" q ON pm."QualificationId" =
q."QualificationId"
    JOIN "WorkType" wt ON p."WorkTypeId" = wt."WorkTypeId" -
- Джойним WorkType
    WHERE p."ProjectId" = :project_id
    """

    result = db.execute(text(query_sql), {"project_id":
project_id}).fetchone()

    if result is None:
        raise HTTPException(status_code=404, detail="Проект не
найден")

    project_data = {
        ...
    }

    return ProjectDetails(**project_data)

```

3.2 Реализация интерфейса пользователя

Интерфейс оконного приложения реализован с использованием технологии WPF. Для организации навигации между различными логическими частями приложения применена архитектура на основе Page и Frame. Это позволяет динамически загружать и переключать содержимое окна.. Для отображения списка проектов применяется компонент DataGrid, который представлен рисунком 4. Код разработанного компонента DataGrid представлен листингом 4 [4].

Название	Описание	Заказчик	Руководитель проекта	Дата заказа	Статус	Действия
Проект1	Описание	Заказчик1	Михаил	01.01.2025	В очереди	 
Проект3	Проект с датой рождения	ООО НовыйЗаказчик	Михаил	15.11.2024	В очереди	 
Проект2		ООО Грибочки	Кирилл Ключанов	01.03.2025	Завершено	 

Рисунок 4 – Вид компонента DataGrid

Листинг 4 – Код верстки компонента DataGrid

```

<!-- Компонент DataGrid для отображения списка проектов -->
<DataGrid x:Name="ProjectsDataGrid" AutoGenerateColumns="False"
CanUserAddRows="False" CanUserReorderColumns="False"
CanUserResizeColumns="False" CanUserSortColumns="False"
CanUserDeleteRows="False" CanUserResizeRows="False"
IsReadOnly="True" Margin="0,0,0,10"
Style="{StaticResource ProjectDataGridStyle}"
<!-- Применяем глобальный стиль для DataGrid -->
    HorizontalAlignment="Center"
    VerticalAlignment="Center"
    HorizontalContentAlignment="Center"
    VerticalContentAlignment="Center"
    MinRowHeight="50">
    <DataGrid.Columns>
    <!-- Используем TemplateColumn, чтобы отобразить кнопку вместо
    простого текста -->
        <DataGridTemplateColumn Header="Название"
IsReadOnly="True" Width="*" HeaderStyle="{StaticResource
CenteredColumnHeaderStyle}">
            <DataGridTemplateColumn.CellTemplate>
                <DataTemplate>
                    <!-- IsEnabled привязан к роли пользователя-->
                    <Button Content="{Binding ProjectName}"
                        Style="{StaticResource
ProjectNameLinkButtonStyle}"
                        Click="ProjectName_Click"
                        Tag="{Binding ProjectNumber}"
                        Margin="10 0 0 0"
                        IsEnabled="{Binding Source={x:Static
api:UserSession.HasManagerAccess}}"/>
                </DataTemplate>
            </DataGridTemplateColumn.CellTemplate>
        </DataGridTemplateColumn>
        <DataGridTextColumn Header="Описание" Binding="{Binding
Description}" IsReadOnly="True" Width="2*"
HeaderStyle="{StaticResource CenteredColumnHeaderStyle}"/>
        <DataGridTextColumn Header="Заказчик" Binding="{Binding
Employer}" IsReadOnly="True" Width="*"
HeaderStyle="{StaticResource CenteredColumnHeaderStyle}"/>

```

```

        <DataGridTextColumn Header="Руководитель проекта"
Binding="{Binding ProjectManagerFullName}" IsReadOnly="True"
Width="*" HeaderStyle="{StaticResource
CenteredColumnHeaderStyle}"/>
        <!-- Используем StringFormat для отображения DateOnly в
формате dd.MM.yyyy -->
        <DataGridTextColumn Header="Дата заказа"
Binding="{Binding ProjectDate, StringFormat=\{0:dd.MM.yyyy\}}"
IsReadOnly="True" Width="*"
HeaderStyle="{StaticResource
CenteredColumnHeaderStyle}"/>
        <!-- Используем TemplateColumn, чтобы отобразить цветную
кнопку/рамку с текстом статуса -->
        <DataGridTemplateColumn Header="Статус"
IsReadOnly="True" Width="*" HeaderStyle="{StaticResource
CenteredColumnHeaderStyle}">
            <DataGridTemplateColumn.CellTemplate>
                <DataTemplate>
                    <!-- Border с фоном, зависящим от значения
Status через StatusToBrushConverter -->
                    <Border Background="{Binding Status,
Converter={StaticResource StatusToBrushConverter}}"
Padding="5,2" CornerRadius="3">
                        <TextBlock Text="{Binding Status}"
Foreground="White" FontWeight="Bold" FontSize="12"
HorizontalAlignment="Center" VerticalAlignment="Center"/>
                    </Border>
                </DataTemplate>
            </DataGridTemplateColumn.CellTemplate>
        </DataGridTemplateColumn>
        <!-- Содержит кнопки "Редактировать" и "Удалить" -->
        <DataGridTemplateColumn Header="Действия"
IsReadOnly="True" Width="*" HeaderStyle="{StaticResource
CenteredColumnHeaderStyle}">
            <DataGridTemplateColumn.CellTemplate>
                <DataTemplate>
                    ...
                </DataTemplate>
            </DataGridTemplateColumn.CellTemplate>
        </DataGridTemplateColumn>
    </DataGrid.Columns>
</DataGrid>

```

3.3 Разграничение прав доступа пользователей

Разграничение прав доступа реализовано разделением пользователей на три роли: администратор, менеджер и гость.

Для осуществления разграничения прав доступа в WPF-приложении реализовано хранение роли пользователя, полученной при аутентификации, и управление видимостью или доступностью элементов интерфейса. Код с методом аутентификации и получением роли пользователя представлен в листинге 5.

Листинг 5 – Метод аутентификации и получения роли пользователя

```
public async Task<AuthResponse?> LoginAsync(string login, string
password)
{
    var loginData = new { Login = login, Password = password };
    var json = JsonSerializer.Serialize(loginData);
    var content = new StringContent(json, Encoding.UTF8,
"application/json");

    var response = await _httpClient.PostAsync("auth/login",
content);
    if (response.IsSuccessStatusCode)
    {
        var responseContent = await
response.Content.ReadAsStringAsync();
        var authResponse =
JsonSerializer.Deserialize<AuthResponse>(responseContent, new
JsonSerializerOptions { PropertyNameCaseInsensitive = true });
        if (authResponse != null && authResponse.Success)
        {
            UserSession.UpdateFromAuthResponse(authResponse);
            SetUserRole(authResponse.RoleName); //
Присваивается роль пользователю
        }
        return authResponse;
    }
    else
    {
        var errorContent = await
response.Content.ReadAsStringAsync();
        try
        {
            ...
        }
        catch
        {
            ...
        }
    }
}
```

3.4 Экспорт и импорт данных

Оконное приложение также предоставляет доступ к импорту в формате .csv данных о проектах организации. Код метода импорта данных ProcessCsvImportAsync представлен листингом 6. Также в оконном приложении присутствует экспорта данных в формате .pdf. Код метода экспорта данных GeneratePdfReport представлен листингом 7.

Листинг 6 – Код метода ProcessCsvImportAsync

```
private async Task ProcessCsvImportAsync(string filePath)
{
    try
    {
        using (TextFieldParser parser = new
            TextFieldParser(filePath, System.Text.Encoding.UTF8))
        {
            parser.TextFieldType = FieldType.Delimited;
            parser.SetDelimiters(";");
            parser.HasFieldsEnclosedInQuotes = false;
            parser.TrimWhiteSpace = true;

            // чтение заголовков
            string[] headers = parser.ReadFields();
            if (headers == null || headers.Length < 7)
            {
                ...
                return;
            }

            // Чтение строк csv-файла
            while (!parser.EndOfData)
            {
                string[] fields = parser.ReadFields();
                if (fields == null || fields.Length <
                    headers.Length)
                {
                    continue;
                }

                // Извлечение значений из каждого столбца
                string name = fields[nameIndex]?.Trim() ??
                    string.Empty;
                string? description = descIndex >= 0 &&
                    descIndex < fields.Length ? fields[descIndex]?.Trim() : null;
```

```

        ...
    }

    // Вывод результата
    MessageBox.Show($"Импорт завершён. Успешно: {importedCount}, Ошибок: {errorCount}.", "Импорт",
    MessageBoxButton.OK, MessageBoxImage.Information);

    // Обновление данных
    await LoadProjectsAsync(...);
}
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка при чтении CSV-файла: {ex.Message}", "Ошибка импорта", MessageBoxButton.OK,
    MessageBoxImage.Error);
}
}

```

Листинг 7 – Код метода создания GeneratePdfReport

```

private void GeneratePdfReport(List<ProjectDetailsModel>
projects, string filePath, int currentPage, int totalPages, int
totalCount, string? statusFilter, string? managerFilter,
DateOnly? startDateFilter, DateOnly? endDateFilter, string?
searchTermFilter)
{
    // Установка лицензии QuestPDF
    QuestPDF.Settings.License = LicenseType.Community;

    QuestPDF.Fluent.Document.Create(container =>
    {
        container.Page(page =>
        {
            // Настройка страницы
            page.Size(PageSizes.A4);
            page.Margin(2, Unit.Centimetre);
            page.DefaultTextStyle(x =>
x.FontSize(9).FontFamily("Segoe UI"));

            // Шапка документа
            page.Header()
                .Text($"Список проектов - Страница {currentPage}
из {totalPages}")
                .SemiBold().FontSize(14);

            // Основное содержание
            page.Content()

```

```

.PaddingVertical(0.5f, Unit.Centimetre)
.Column(column =>
{

    // Таблица с данными проектов
    column.Item()
        .Element(SetTableStyle)
        .Table(table =>
        {
            // Настройка столбцов таблицы
            table.ColumnsDefinition(columns =>
            {
                ...
            });

            // Заголовки таблицы
            table.Header(header =>
            {
                ...
            });

            // Построчное заполнение таблицы
            foreach (var project in projects)
            {
                ...
            }
        });
    });

    // Подвал документа
    page.Footer()
        .AlignCenter()
        .Text($"Всего проектов на этой странице:
{projects.Count} | Всего проектов: {totalCount} | Страница
{currentPage} из {totalPages}");
    });
})
// Генерация pdf-файла
.GeneratePdf(filePath);
}

```

4 Тестирование и отладка программного обеспечения

4.1 Структурное тестирование

Во время курсового проектирования проведено тестирование методом белого ящика для методов Web-API, результаты представлены в таблице 1 [1].

Таблица 1 – Результаты тестирования конечной точки создание заметки

Действие	Ожидаемый результат	Фактический результат
1) Убедиться, что проект с ID 0 не существует 2) Выполнить GET: project/0	Получение HTTP-ответа 404 Not Found. Тело ответа содержит информацию о том, что проект не найден	Совпадает с ожидаемым
1) Убедиться, что проект с ID 2 существует 2) Выполнить GET: project/2	Получение HTTP-ответа 200 OK. Тело ответа содержит детали проекта (ID, название, описание, заказчика, руководителя проекта, дату заказа и т.д.)	Совпадает с ожидаемым
1) Убедиться, что существуют проекты, удовлетворяющие статус фильтру «Активен» 2) Выполнить POST: projects/details запрос в API с телом: { "status": "Активен" }	Получение HTTP-ответа 200 OK. Тело ответа содержит список проектов, общее количество и информацию о пагинации.	Совпадает с ожидаемым
1) Убедиться, что проект с ID 30 можно удалить 2) Выполнить DELETE: /projects/30	Получение HTTP-ответа 200 OK. Тело ответа содержит информацию о том, что проект успешно удалён	Совпадает с ожидаемым

4.2 Функциональное тестирование

Во время курсового проектирования проведено функциональное тестирование оконного приложения методом черного ящика, результаты тестирования представлены в таблице 2.

Таблица 2 – Результаты тестирования приложения методом черного ящика

Действие	Ожидаемый результат	Фактический результат
Нажать на кнопку «Войти как гость»	Переход на экран отображения списка проектов, все кнопки взаимодействия («Создать проект», «Удалить», «Редактировать проект» и т.д.) отключены	Совпадает с ожидаемым
Нажать на кнопку «Удалить» (корзина) и подтвердить действие	Таблица обновляется и проект больше не отображается	Совпадает с ожидаемым
Нажать на кнопку сортировки проектов по дате по убыванию	Проекты сортируются от самой поздней даты к самой ранней	Совпадает с ожидаемым
Применить фильтрацию проектов по статусу «Активен»	Список проектов обновляется, показывая проекты только со статусом «Активен»	Совпадает с ожидаемым
Нажать на кнопку «Создать проект»	Переход на страницу создания проекта с полями для ввода данных	Совпадает с ожидаемым
Нажать на название проекта	Переход на страницу, где отображаются все детали проекта	Совпадает с ожидаемым
Нажать на кнопку «Следующая»	Загружается следующая страница проектов, информация о номере страницы изменяется	Совпадает с ожидаемым
Оставить поле ввода «Название» пустым на странице создания проекта	Появляется сообщение об ошибке «Все обязательные поля должны быть заполнены»	Совпадает с ожидаемым
Нажать на кнопку «Экспортировать в pdf-файл» и указать путь для сохранения файла	Создаётся PDF-файл с таблицей проектов текущей страницы	Совпадает с ожидаемым
Нажать на кнопку «Импортировать из csv-файла» и выбрать корректный файл для импорта	В таблицу добавляются проекты из импортированного файла.	Совпадает с ожидаемым

По результатам тестирования можно сделать вывод, что разработанное ПО работает корректно и согласно ожиданиям.

5 Инструкция по эксплуатации программного обеспечения

5.1 Установка программного обеспечения

Для функционирования подсистемы на стороне сервера достаточны следующие программные и технические средства:

- ОС Ubuntu Server версии 22.04 или аналоги;
- процессор с частотой 2 ГГц и выше;
- свободная оперативная память объемом 4 ГБ;
- дополнительные компоненты: Python 3.12, Uvicorn, Pydantic, FastAPI для работы API, PostgreSQL 16, pgAdmin 4.

Для установки серверной части необходимо выполнить следующую последовательность действий:

- Установить PostgreSQL 16;
- Установить и запустить pgAdmin 4, осуществить подключение к серверу и выполнить SQL-скрипт из папки Scripts на флешке.
- Скопировать файлы из папки FastAPI на флешке на свой ПК или удалённый сервер.
- Настроить виртуальное окружение Python
- В файле database.py настроить строку подключения к БД (листинг 8).

Листинг 8 – Строка подключения database.py

```
DATABASE_URL =  
"postgresql://user_name:password@host/your_database"
```

Для функционирования оконного приложения на стороне клиента достаточны следующие программные и технические средства:

- ОС Windows 10 и выше.
- процессор с частотой 2 ГГц и выше;
- оперативная память: минимум 2 ГБ;

- постоянное интернет-подключение.
- Дополнительные компоненты: среда обработки Visual Studio 2022, готовый исполняемый файл IntiClient.exe

В оконном приложении используются следующие данные для авторизации:

- логин – Login123;
- пароль – qwerty123.

5.2 Инструкция по работе

После открытия оконного приложения пользователь попадает на страницу авторизации, где ему необходимо ввести учетные данные для входа в систему. Страница авторизации представлена на рисунке 4. После авторизации у пользователя определяется его роль в системе: администратор, менеджер, гость. В зависимости от роли пользователь имеет разные права на доступ к функциональности системы.

Рисунок 4 – Вид страницы авторизации

После успешной авторизации пользователь попадает на страницу управления проектами компании (рисунок 5), на которой отображаются все существующие проекты. Имеется возможность поиска проекта по его названию, фильтрации по статусу, руководителю проекта и дате, сортировки проектов по дате, переходить по страницам с проектами.

Проекты компании

Управление проектами

Иванов (Администратор)

Импортировать из csv-файла

Экспортировать в pdf-файл

+ Добавить проект

-- Выйти

Список проектов

Всего проектов: 37 | Показано: 15

Поиск по названию проекта:

Сорт. по дате I

Сорт. по дате I

Сброс

Фильтры

Название	Описание	Заказчик	Руководитель проекта	Дата заказа	Статус	Действия
Проект1	Описание	Заказчик1	Михаил	01.01.2025	В ожидании	 
Проект3	Проект с датой рождения	ООО НовыйЗаказчик	Михаил	15.11.2024	В ожидании	 
Проект2		ООО Грибочки	Кирилл Ключанов	01.03.2025	Завершено	 
Проект1	Описание проекта 1	ООО Грибочки	Михаил	01.03.2025	Активен	 
Проект3	Проект с датой рождения	ООО НовыйЗаказчик	Михаил	15.11.2024	В ожидании	 
Проект1	Описание проекта 1	ООО Грибочки	Михаил	01.03.2025	Активен	 
dsd		ss	Михаил	06.12.2025	Завершено	 
sadaas	asdasd	asdasd	Кирилл Ключанов	03.12.2025	Активен	 
sda	asd	asd	Кирилл Ключанов	04.12.2025	Завершено	 
asd	asd	asdsad	Михаил	01.12.2025	Завершено	 
asd	asd	asdsad	Кирилл Ключанов	04.12.2025	Завершено	 
Привет	Как дела	Я	Кирилл Ключанов	05.12.2025	Завершено	 
Dsadas	asdasd	asdasd	Кирилл Ключанов	02.12.2025	Активен	 
Name	Description	Employer	Михаил	05.12.2025	В ожидании	 
Name	Description	Employer	Михаил	05.12.2025	В ожидании	 

Предыдущая

Стр. 1 из 3

Следующая

Рисунок 5 –Вид страницы управления проектами

Для создания проекта организации необходимо нажать на кнопку «+ Добавить проект», при нажатии осуществляется переход на страницу с полями ввода «Название», «Описание», «Заказчик», с комбинированными списками «Руководитель проекта», «Тип работ», «Статус» и датапикером «Дата заказа проекта». При добавлении проекта автоматически

Для получения подробной информации о проекте пользователю необходимо нажать на название проекта, после чего осуществляется переход на страницу с подробностями проекта (рисунок 6).

Проект1

Описание:	Описание
Заказчик:	Заказчик1
Руководитель:	Михаил
Основная квалификация руководителя:	Инженер-изыскатель
Тип работ:	Заполнение сметы
Дата заказа проекта:	01.01.2025
Статус:	В очереди
ID Проекта:	45

[Редактировать проект](#) [Удалить проект](#) [Назад](#)

Рисунок 6 –Вид страницы подробностей проекта

На странице подробностей проекта отображаются данные о проекте, руководителе и его квалификации. При нажатии кнопки «Удалить проект» появляется диалоговое окно с подтверждением удаления проекта. При нажатии кнопки «Редактировать проект» осуществляется переход на страницу редактирования проекта (рисунок 7).

Редактирование проекта

Название проекта*

Проект1

Описание

Описание

Заказчик*

Заказчик1

Руководитель проекта*

Михаил

Тип работ*

Заполнение сметы

Дата заказа проекта*

01.01.2025

Статус*

В очереди

[Сохранить](#) [Отмена](#)

Рисунок 7 –Вид страницы редактирования проекта

ЗАКЛЮЧЕНИЕ

В рамках курсового проекта была успешно разработана подсистема управления проектами для организации. Созданное оконное приложение взаимодействует с серверной частью, заменяя устаревшие ручные методы работы на централизованную цифровую систему, что напрямую решает такие задачи как: сокращение временных затрат, минимизация ошибок и обеспечение оперативного доступа к информации о проектах, их статусах, руководителях и типах работ.

Цель проекта: разработка программного решения для автоматизации учёта и управления проектами достигнута.

В процессе выполнения решены все ключевые задачи:

- проведен анализ предметной области;
- определены требования к разрабатываемому ПО;
- спроектирована архитектура системы, включающая серверную часть, базу данных и клиентское оконное приложение;
- реализована серверная часть, обеспечивающая выполнение бизнес-логики приложения и разграничение прав доступа;
- разработано оконное приложение для менеджеров компании;
- обеспечено удобство взаимодействия пользователя с приложением за счёт оптимизации навигации и логики интерфейса;
- проведено тестирование функциональности.

Итогом проделанной работы является разработанная подсистема. Выполнение проекта от анализа требований до тестирования позволило пройти полный жизненный цикл разработки ПО. Приобретенные практические навыки в проектировании клиент-серверной архитектуры, взаимодействии с базой данных, создании API и разработке пользовательского интерфейса, а также в их интеграции, составляют ценный опыт, подтверждающий готовность к реализации реальных проектов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бек, К. Экстремальное программирование: разработка через тестирование. – Санкт-Петербург : Питер, 2021. – 224 с. – URL: <https://ibooks.ru/bookshelf/376974/reading> (дата обращения: 01.11.2025). – Режим доступа: для зарегистрир. пользователей. – Текст : электронный.
2. Гагарина, Л. Г. Технология разработки программного обеспечения : учебное пособие / Л. Г. Гагарина, Е. В. Кокорева, Б. Д. Сидорова-Виснадул ; под ред. Л. Г. Гагариной. – Москва : ФОРУМ : ИНФРА-М, 2025. – 400 с. – URL: <https://znanium.ru/catalog/product/2178802> (дата обращения: 12.11.2025). – Режим доступа: по подписке. – Текст : электронный.
3. Мартишин, С. А. Базы данных. Практическое применение СУБД SQL и NoSQL-типа для проектирования информационных систем : учебное пособие / С. А. Мартишин, В. Л. Симонов, М. В. Храпченко. – Москва : ФОРУМ : ИНФРА-М, 2024. – 368 с. – URL: <https://znanium.ru/catalog/product/2096940> (дата обращения: 23.11.2025). – Режим доступа: по подписке. – Текст : электронный.
4. Тидвелл, Д. Разработка интерфейсов. Паттерны проектирования. 3-е изд. – Санкт-Петербург : Питер, 2022. – 560 с. – URL: <https://ibooks.ru/bookshelf/386796/reading> (дата обращения: 23.11.2025). – Режим доступа: для зарегистрир. пользователей. – Текст : электронный.
5. Федорова, Г. Н. Разработка, внедрение и адаптация программного обеспечения отраслевой направленности : учебное пособие. – Москва : КУРС : ИНФРА-М, 2024. – 336 с. – URL: <https://znanium.ru/catalog/product/2083407> (дата обращения: 24.11.2025). – Режим доступа: по подписке. – Текст : электронный.